

ANÁLISE - SPRINT 3

1. Dimensionalidade: o que muda e o que não muda

O Experimento 1 mostra uma separação importante. Variar `d_out` (8, 16, 32) altera o tamanho do vetor de contexto de cada token, mas nunca a quantidade de tokens: a saída é sempre `(n_tokens, d_out)`. Já variar `num_heads` com `d_out` fixo em 32 **não muda nada no formato da saída**, porque as cabeças repartem o mesmo espaço: `head_dim = d_out / num_heads`.

A conclusão é que o número de cabeças não é um parâmetro de capacidade, e sim de *particionamento*. Com 8 cabeças de 4 dimensões, o modelo tem a mesma quantidade de parâmetros que com 1 cabeça de 32, só que distribuídos em oito subespaços independentes, cada um capaz de aprender um tipo diferente de relação entre tokens. Existe, portanto, um compromisso: muitas cabeças dão mais diversidade de padrões, mas cada uma enxerga um espaço mais estreito. O `AssertionError` com `num_heads=5` confirma a restrição estrutural: como a divisão dos pesos é feita por `view`, `d_out` precisa ser divisível pelo número de cabeças. Não é uma escolha de estilo, é uma exigência de formato do tensor.

2. Self-Attention × Multi-Head

A diferença média de 0,0453 entre as duas saídas confirma que os módulos produzem representações distintas, mas é preciso ter cuidado com a interpretação: **os pesos não foram treinados**. Com inicializações aleatórias diferentes, qualquer par de módulos daria uma diferença dessa ordem. O número mostra que as arquiteturas não são equivalentes ponto a ponto, não que a multi-head seja superior. A vantagem real da multi-head só aparece depois do treino, quando cada cabeça se especializa. O que já dá para afirmar agora, estruturalmente, é que a versão multi-head acrescenta a projeção de saída (`out_proj`), ausente na atenção de cabeça única, e que ela calcula todas as cabeças numa única multiplicação em lote, em vez de uma por vez.

3. A escala por $\sqrt{d_k}$

Este é o resultado mais informativo do conjunto. Com `d_k=64`, a atenção sem escala concentrou-se mais (maior peso 0,2030, entropia 2,1414), enquanto com

escala ficou praticamente uniforme (maior peso 0,1210, entropia 2,1963, contra o máximo teórico de $2,1972 = \ln 9$).

A direção é exatamente a que o livro prevê: dividir por $\sqrt{d_k}$ comprime os produtos escalares para perto de zero e impede que a softmax colapse em um único token. O efeito ainda é modesto porque $d_k=64$ é pequeno perto das dimensões reais de um LLM, e porque os embeddings não treinados têm variância baixa. Numa frase: **a escala não melhora a atenção em si, ela protege o gradiente**. Sem ela, em dimensões grandes, a softmax entra numa região quase plana e o aprendizado praticamente para.

4. A máscara causal

O Experimento 3 é o mais conclusivo, porque a máscara é determinística e independe de treino. Os dois heatmaps mostram o comportamento esperado, e os valores da frase curta são reveladores:

- "O" (1ª posição): peso 1,00 em si mesma, já que não há nada antes;
- "gato": 0,50 e 0,50, dividindo entre "O" e ela mesma;
- "dorme": aproximadamente 0,33 em cada uma das três palavras.

Esse padrão vale uma observação na análise: os valores são quase perfeitamente uniformes (1 , $\frac{1}{2}$, $\frac{1}{3}$) porque, sem treino, as pontuações são praticamente indistinguíveis entre si. Depois do treino é que essas linhas passam a ter picos em tokens relevantes. Ou seja, **a estrutura triangular é a mecânica; a variação dentro de cada linha é o aprendizado**. O ponto crucial é que cada linha continua somando 1 mesmo após o mascaramento. É isso que garante que o vetor de contexto permaneça uma média ponderada legítima, apenas restrita ao passado, sem vazamento de informação futura.

5. Integração com a Sprint 2 e limitações

Os quatro mecanismos aceitaram sem alteração o tensor [2, 4, 256] construído no pipeline da Sprint 2, o que valida que a saída daquela sprint é de fato a entrada desta.

6. Custo computacional e o gargalo quadrático.

A matriz de atenção tem formato $T \times T$, então o custo cresce com o quadrado do número de tokens, não linearmente. Dobrar a janela de contexto quadruplica

memória e cálculo nessa etapa. Isso conecta direto com a conclusão da Sprint 2 sobre contextos grandes exigirem mais dados, e explica por que a janela de contexto é um dos limites práticos mais caros de um LLM.

7. Por que as duas formas de mascarar dão o mesmo resultado.

Implementamos os dois caminhos em 3,3. Vale registrar que mascarar com `-inf` antes da softmax é matematicamente idêntico a zerar depois e renormalizar, porque a softmax é invariante a esse tipo de reescala. A segunda versão só evita uma passagem extra pelos dados e deixa claro que não há vazamento em nenhum dos dois casos.

8. A eficiência da divisão de pesos.

O wrapper faz uma multiplicação de matriz por cabeça, em sequência; a versão eficiente faz uma projeção só e fatia o resultado. As duas são conceitualmente equivalentes, mas a segunda aproveita muito melhor a GPU, que é otimizada justamente para poucas operações grandes em vez de muitas pequenas. Se vocês tiverem medido o tempo das duas, o número entra aqui.

9. O papel do dropout, e por que ele ficou desligado.

Todos os experimentos usaram taxa 0,0, o que foi a escolha certa: o dropout é aleatório e só atua em modo de treino, então mantê-lo ligado tornaria os resultados irreprodutíveis. É bom deixar explícito que ele existe nas classes, mas só passa a fazer diferença a partir do pré-treinamento, como regularizador.

10. Reprodutibilidade.

Usamos as mesmas seeds do livro (123 e 789), o que permitiu conferir valor por valor com o texto original. Isso é uma forma simples e forte de validação: confirma que a implementação está correta, e não apenas que roda sem erro.

Vale registrar explicitamente as limitações desta etapa: todos os resultados vêm de pesos aleatórios, com dropout em 0,0 e sem nenhuma iteração de treinamento. Por isso, os padrões de atenção observados são estruturais, e não semânticos. A análise do *conteúdo* da atenção, isto é, quais palavras o modelo aprende a

relacionar, só será possível após o pré-treinamento do capítulo 5. O que esta sprint entrega, e que a Sprint 4 vai consumir, é um módulo de atenção correto em formato, causal por construção e pronto para ser empilhado nos blocos transformer.